

在 AMD Radeon GPU 上运行 Chatterbox TTS 语音合成服务

文字转语音（Text-to-Speech，TTS）技术可以将文字自动转换为自然流畅的语音，广泛应用于有声书制作、虚拟助手、无障碍辅助等场景。

[Chatterbox TTS Server](#) 是一个基于 Chatterbox 模型的**开源自托管语音合成服务器**，提供友好的 Web UI 操作界面与 OpenAI 兼容的 REST API 接口，支持多种预置声音和声音克隆功能，可在 NVIDIA（CUDA）、AMD（ROCm）和 CPU 上加速运行。

本教程带你**在 AMD Radeon GPU 云平台（radeon.anruicloud.com）上，完全在 Jupyter Notebook 内完成服务安装、启动与语音生成**，无需额外的终端窗口或端口映射。

关于 AMD OneClick 平台：该平台提供预装 ROCm 的 Docker 容器环境，通过 Jupyter Lab 访问。由于平台只对外暴露 Jupyter 的 8888 端口，本教程将通过 Python API 调用的方式在 Notebook 内直接生成并播放语音，而非通过浏览器访问 Web UI。

你会学到

学完本教程，你将能够：

1. 在 AMD Radeon GPU（ROCm 软件栈）上验证 GPU 可用性。
2. 正确安装 Chatterbox TTS Server 及其 AMD ROCm 依赖。
3. 在 Notebook 中以后台进程方式启动 TTS 服务器。
4. 通过 OpenAI 兼容 API 生成语音并在 Notebook 中直接播放。
5. 使用 `ipywidgets` 构建一个交互式语音合成控制面板。

前置条件

本教程在以下环境开发并验证。

平台

- **AMD OneClick Cloud** (radeon.anruicloud.com)
- 基础镜像：`rocm/vllm-dev:rocm7.2.1_navi_ubuntu24.04_py3.12_pytorch_2.9_vllm_0.16.0`
- GitHub Repo URL: <https://github.com/devnen/Chatterbox-TTS-Server.git> (创建template时填写)

硬件

- **AMD Radeon GPU**：本教程在 **AMD RX 7900 系列**（RDNA3，gfx1100）上测试。其他受支持型号包括 RX 7900 XT / XTX / GRE，RX 9070 / 9070 XT（RDNA4）等。

软件

- **ROCm 7.1.2 / 7.2+**：平台镜像已预装，无需手动安装。
- **PyTorch 2.9（ROCm 版）**：平台镜像已预装，无需重新安装。
- **Python 3.12**：平台镜像默认 Python 版本。

网络说明

该平台容器内**无法直接访问 GitHub 和 huggingface.co**，本教程已针对这一限制提供了以下解决方案：

- 使用 hf-mirror.com 作为 HuggingFace 镜像下载模型。
- `chatterbox-v2` 引擎通过预先上传的 ZIP 包进行安装(点击下载[chatterbox-v2-add-disc.zip](#))。

一、验证 AMD GPU 与 ROCm 环境

在正式安装和运行任何 AI 模型之前，最重要的一步是确认 GPU 驱动和 ROCm 软件栈工作正常。

`rocminfo` 是 AMD 官方提供的工具，可以列出系统中所有被 ROCm 识别的 GPU 信息，包括 GPU 架构代号（如 gfx1100 对应 RDNA3）、显存大小等。如果这个命令没有输出 GPU 信息，说明 ROCm 驱动安装存在问题。

```
In [1]: import sys
print("Python 版本:", sys.version)

!rocminfo | grep -E 'Name|Marketing Name|gfx|Vendor'
```

```
Python 版本: 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0]
Name: AMD EPYC 9334 32-Core Processor
Marketing Name: AMD EPYC 9334 32-Core Processor
Vendor Name: CPU
Name: AMD EPYC 9334 32-Core Processor
Marketing Name: AMD EPYC 9334 32-Core Processor
Vendor Name: CPU
Name: gfx1100
Marketing Name: AMD Radeon Graphics
Vendor Name: AMD
Name: amdgcN-amd-amdhsa--gfx1100
Name: amdgcN-amd-amdhsa--gfx11-generic
```

`rocm-smi` 是另一个常用工具，类似 NVIDIA 的 `nvidia-smi`，可以实时查看 GPU 的温度、功耗、显存占用和利用率等运行状态。

In [2]: `!rocm-smi`

```
===== ROCm System Management Interface =====
=====
===== Concise Info =====
=====
Device Node IDs          Temp    Power Partitions          SCLK  MCLK  Fan
Perf PwrCap VRAM% GPU%
          (DID,      GUID) (Edge) (Avg) (Mem, Compute, ID)
=====
0      3      0x744b,  49884  28.0°C  15.0W  N/A, N/A, 0          0Mhz  96Mhz  20.0%
auto  241.0W  0%      0%
=====
=====
===== End of ROCm SMI Log =====
=====
```

最后，验证 PyTorch 是否能识别到 AMD GPU。

这里有一个初学者常见的疑问：**为什么 AMD GPU 要用 `torch.cuda` 这个名字？** 这是因为 AMD ROCm 在 PyTorch 层面实现了与 CUDA 兼容的 API（底层使用 HIP 运行时），因此 `torch.cuda.is_available()`、`torch.cuda.get_device_name()` 等函数在 AMD GPU 上同样适用。这是正常设计，不是错误。

In [3]: `import torch`

```
print("PyTorch 版本 :", torch.__version__)
print("ROCm GPU 可用 :", torch.cuda.is_available())
print("GPU 数量      :", torch.cuda.device_count())

if torch.cuda.is_available():
    print("GPU 型号      :", torch.cuda.get_device_name(0))
    vram_gb = torch.cuda.get_device_properties(0).total_memory / 1024**3
    print(f"显存总量      : {vram_gb:.1f} GB")
    print("\n✅ GPU 环境正常，可以继续后续步骤。")
else:
    print("\n⚠️ 未检测到 ROCm GPU，请检查平台是否已分配 GPU 资源。")
```

```
PyTorch 版本 : 2.9.1+gitff65f5b
ROCm GPU 可用 : True
GPU 数量      : 1
GPU 型号      : AMD Radeon Graphics
显存总量      : 48.0 GB
```

✅ GPU 环境正常，可以继续后续步骤。

二、设置 HuggingFace 镜像

Chatterbox TTS 使用的模型权重托管在 HuggingFace Hub 上，首次运行时需要从网络下载。然而，AMD OneClick 平台的容器内无法直接连接 `huggingface.co`。

解决方案是将 `HF_ENDPOINT` 环境变量设置为国内镜像站地址。HuggingFace 的 Python 库会自动读取这个变量，并将所有模型下载请求重定向到镜像站。

重要：这个设置必须在启动 TTS 服务器之前完成，否则服务器启动时会尝试连接被墙的官方地址，导致下载超时失败。

```
In [4]: import os

os.environ["HF_ENDPOINT"] = "https://hf-mirror.com"
print("HF_ENDPOINT 已设置为:", os.environ["HF_ENDPOINT"])
```

HF_ENDPOINT 已设置为: https://hf-mirror.com

设置好后，先测试一下镜像站是否可以正常连接，避免后续下载模型时才发现网络有问题。

```
In [5]: import urllib.request

print("正在测试镜像站连通性...")
try:
    urllib.request.urlopen("https://hf-mirror.com", timeout=10)
    print("✅ hf-mirror.com 可访问，后续模型下载将使用此镜像站。")
except Exception as e:
    print(f"⚠️ 镜像站连接失败: {e}")
    print("🔍 请检查网络连接，或联系平台管理员。")
```

正在测试镜像站连通性...

✅ hf-mirror.com 可访问，后续模型下载将使用此镜像站。

三、获取 Chatterbox TTS Server 代码

由于平台容器无法访问 GitHub，仓库代码应该已经预置在 `/workspace/repo` 目录下。运行下方代码检查仓库是否完整。

如果仓库不存在，代码会先尝试网络克隆，如果克隆失败，会提示手动上传的步骤。

```
In [6]: import os

REPO_DIR = "/workspace/repo"

key_files = ["server.py", "requirements-rdna4.txt", "requirements-rdna4-init.txt"]
missing = [f for f in key_files if not os.path.exists(os.path.join(REPO_DIR, f))]

if not missing:
    print(f"✅ 仓库完整，路径: {REPO_DIR}")
else:
    print(f"⚠️ 以下文件缺失: {missing}")
```

✅ 仓库完整，路径: /workspace/repo

查看仓库中包含哪些文件，重点关注几个与安装相关的文件：

- `requirements-rdna4-init.txt` — RDNA4 专用 PyTorch ROCm 7.2 轮子列表

- requirements-rdna4.txt — 服务器其余 Python 依赖
- chatterbox-v2-add-disc.zip — 离线安装用的引擎源码包
- server.py — TTS 服务器主程序
- config.yaml — 服务器配置文件

```
In [7]: import os

files = sorted([f for f in os.listdir(REPO_DIR) if not f.startswith('.')])
print(f"仓库文件 (共 {len(files)} 个): ")
for f in files:
    print(f" {f}")
```

仓库文件 (共 51 个):

Chatterbox_TTS_Colab_Demo.ipynb
Dockerfile
Dockerfile.cpu
Dockerfile.cu128
Dockerfile.cu130
Dockerfile.rdna4
Dockerfile.rocm
Dockerfile.strixhalo
LICENSE
README.md
README_CUDA128.md
README_Colab.md
__pycache__
chatterbox-rocm-amd.ipynb
chatterbox-rocm-amd_v2.ipynb
chatterbox-v2-add-disc.zip
config.py
config.yaml
docker-compose-cpu.yml
docker-compose-cu128.yml
docker-compose-cu130.yml
docker-compose-rdna4.yml
docker-compose-rocm.yml
docker-compose-strixhalo.yml
docker-compose.yml
documentation.md
download_model.py
engine.py
logs
model_cache
models.py
outputs
reference_audio
requirements-nvidia-cu128.txt
requirements-nvidia-cu130.txt
requirements-nvidia.txt
requirements-rdna4-init.txt
requirements-rdna4.txt
requirements-rocm-init.txt
requirements-rocm.txt
requirements-strixhalo-init.txt
requirements-strixhalo.txt
requirements.txt
server.py
start.bat
start.py
start.sh
static
ui
utils.py
voices

如果仓库不存在, 尝试从 GitHub 克隆:

```
In [8]: import os

if not os.path.exists(os.path.join(REPO_DIR, "server.py")):
    print("仓库不存在, 尝试从 GitHub 克隆...")
    !git clone https://github.com/devnen/Chatterbox-TTS-Server.git {REPO_DIR}
else:
    print("✅ 仓库已存在, 跳过克隆。")
```

✅ 仓库已存在, 跳过克隆。

四、安装依赖

安装分三步完成, 每步解决不同层次的依赖。

4.1 关于 PyTorch — 为什么跳过步骤 1

`requirements-rdna4-init.txt` 文件里列出了 PyTorch 2.9.1 的 ROCm 7.2.3 版本轮子 (.whl 文件)。在普通的裸机 Linux 环境下, 你需要先下载并安装这些轮子, 才能让 PyTorch 支持 AMD GPU。

但在本平台上, **基础镜像已经预装好了 PyTorch 2.9 + ROCm**, 如果强行重装, 反而会因为 Python 版本不匹配 (轮子是为 Python 3.10 编译的, 而镜像用的是 Python 3.12) 报错。所以这一步可以直接跳过。

```
In [9]: import torch

print("跳过 requirements-rdna4-init.txt 的安装 (平台镜像已预装)")
print(f"\n当前 PyTorch : {torch.__version__}")
print(f"ROCM GPU 可用 : {torch.cuda.is_available()}")
print("\n✅ PyTorch 就绪, 直接进入下一步。")
```

跳过 requirements-rdna4-init.txt 的安装 (平台镜像已预装)

当前 PyTorch : 2.9.1+gitff65f5b
ROCM GPU 可用 : True

✅ PyTorch 就绪, 直接进入下一步。

4.2 安装服务器依赖

`requirements-rdna4.txt` 包含了 Chatterbox TTS Server 运行所需的其他 Python 库, 例如 FastAPI (Web 框架)、uvicorn (HTTP 服务器)、soundfile (音频处理) 等。

这个文件经过特别设计, 不会触碰已安装的 PyTorch, 安装过程是安全的。

```
In [10]: print("正在安装服务器依赖 (requirements-rdna4.txt) ...")
print("这可能需要 2-5 分钟, 请耐心等待。")

!pip install -r {REPO_DIR}/requirements-rdna4.txt -q
```

```
print("\n✅ 安装完成")
```

正在安装服务器依赖 (requirements-rdna4.txt) ...

这可能需要 2-5 分钟, 请耐心等待。

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

onnx 1.16.0 requires protobuf>=3.20.2, but you have protobuf 3.19.6 which is incompatible.

google-api-core 2.30.0 requires protobuf<7.0.0,>=4.25.8, but you have protobuf 3.19.6 which is incompatible.

googleapis-common-protos 1.73.0 requires protobuf!=4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<7.0.0,>=3.20.2, but you have protobuf 3.19.6 which is incompatible.

grpcio-reflection 1.78.0 requires protobuf<7.0.0,>=6.31.1, but you have protobuf 3.19.6 which is incompatible.

grpcio-tools 1.78.0 requires protobuf<7.0.0,>=6.31.1, but you have protobuf 3.19.6 which is incompatible.

ray 2.54.0 requires protobuf>=3.20.3, but you have protobuf 3.19.6 which is incompatible.

vllm 0.16.1.dev0+g89a77b108.d20260317.rocm721 requires protobuf!=6.30.*,!=6.31.*,!=6.32.*,!=6.33.0.*,!=6.33.1.*,!=6.33.2.*,!=6.33.3.*,!=6.33.4.*,>=5.29.6, but you have protobuf 3.19.6 which is incompatible.

vllm 0.16.1.dev0+g89a77b108.d20260317.rocm721 requires tokenizers>=0.21.1, but you have tokenizers 0.20.3 which is incompatible.

vllm 0.16.1.dev0+g89a77b108.d20260317.rocm721 requires transformers<5,>=4.56.0, but you have transformers 4.46.3 which is incompatible.

[notice] A new release of pip is available: 26.0.1 -> 26.1.2

[notice] To update, run: `pip install --upgrade pip`

✅ 安装完成

4.3 安装 Chatterbox TTS 核心引擎

这是最关键也最容易出问题的一步。需要安装以下三个包：

- **chatterbox-v2**：核心 TTS 模型库，包含语音合成逻辑。必须加 `--no-deps` 标志安装——因为它在 PyPI 上声明依赖的是 CPU 版 PyTorch，如果让 pip 自动处理依赖，ROCm 版 PyTorch 会被替换成 CPU 版，导致 GPU 失效。
- **s3tokenizer==0.3.0**：语音 tokenizer，Chatterbox 用它将音频转成 token 序列。固定版本避免 API 不兼容。
- **onnx==1.16.0**：用于运行 Chatterbox 中的 ONNX 格式子模型。

代码会先尝试从 GitHub 安装，如果网络不通则自动切换到仓库中预置的 ZIP 包。

```
In [11]: print("方法 A: 尝试从 GitHub 安装 chatterbox-v2...")

!pip install --no-deps "git+https://github.com/devnen/chatterbox-v2.git@master" \
    s3tokenizer==0.3.0 onnx==1.16.0 "protobuf>=3.20.1,<5" -q

# 记录安装结果, 供方法 B 判断是否需要执行
```



```
import subprocess, sys
_git_ok = subprocess.run(
    [sys.executable, "-c", "from chatterbox.tts import ChatterboxTTS"],
    capture_output=True
).returncode == 0
print("✅ 方法 A 成功" if _git_ok else "❌ 方法 A 失败, 将使用方法 B。")
```

方法 A: 尝试从 GitHub 安装 chatterbox-v2...

error: subprocess-exited-with-error

× git clone --filter=blob:none --quiet <https://github.com/devnen/chatterbox-v2.git> /tmp/pip-req-build-wwgn00e8 did not run successfully.

| exit code: 128

↳ [1 lines of output]

fatal: unable to access 'https://github.com/devnen/chatterbox-v2.git/': server certificate verification failed. CAfile: none CRLfile: none

[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.

[notice] A new release of pip is available: 26.0.1 -> 26.1.2

[notice] To update, run: pip install --upgrade pip

ERROR: Failed to build 'git+https://github.com/devnen/chatterbox-v2.git@master' when git clone --filter=blob:none --quiet https://github.com/devnen/chatterbox-v2.git /tmp/pip-req-build-wwgn00e8

❌ 方法 A 失败, 将使用方法 B。

如果方法 A (GitHub) 失败, 改为从仓库下载安装包 [chatterbox-v2-add-disc.zip](#), 上传到当前目录。

```
In [12]: import subprocess, sys, os, zipfile
EXTRA_PKGS = ["protobuf>=3.20.1", "s3tokenizer==0.3.0", "onnx==1.16.0"]
if not _git_ok:
    print("方法 B: 从本地 ZIP 包安装...")

    zip_candidates = [
        os.path.join(REPO_DIR, "chatterbox-v2-add-disc.zip"),
        "/workspace/chatterbox-v2.zip",
        "/tmp/chatterbox-v2.zip",
    ]
    zip_path = next((p for p in zip_candidates if os.path.exists(p)), None)

    if not zip_path:
        print("❌ 未找到 ZIP 文件。")
        print("  请在本地下载 https://github.com/devnen/chatterbox-v2/archive/refs/
        print(f"  并上传到{REPO_DIR}, 再重新运行本单元格。")
        print("  或上传到/workspace/, 重命名为 chatterbox-v2.zip, 再重新运行本单元格。")
    else:
        print(f"  找到 ZIP: {zip_path}")
        extract_dir = "/tmp/chatterbox-v2-src"
        os.makedirs(extract_dir, exist_ok=True)
        with zipfile.ZipFile(zip_path) as zf:
            zf.extractall(extract_dir)

        install_dir = extract_dir
```

```

for root, dirs, files in os.walk(extract_dir):
    if "pyproject.toml" in files or "setup.py" in files:
        install_dir = root
        break

result = subprocess.run(
    [sys.executable, "-m", "pip", "install", "--no-deps", install_dir] + EX
    capture_output=True, text=True
)
_zip_ok = result.returncode == 0
print("✅ 方法 B 成功" if _zip_ok else f"❌ 方法 B 失败: {result.stderr[-300]}")
else:
    print("已通过方法 A 安装成功, 跳过方法 B。")

```

方法 B: 从本地 ZIP 包安装...

找到 ZIP: /workspace/repo/chatterbox-v2-add-disc.zip

✅ 方法 B 成功

安装完成后, 尝试导入模块来验证安装是否正确。如果出现报错, 请看下一步的修复说明。

```

In [13]: print("验证 chatterbox 是否可正常导入...")

!python -c "from chatterbox.tts import ChatterboxTTS; print('✅ chatterbox 导入成功'

print("\n💡 如果出现 protobuf 相关报错, 运行下一个单元格修复。")

```

验证 chatterbox 是否可正常导入...

✅ chatterbox 导入成功!

💡 如果出现 protobuf 相关报错, 运行下一个单元格修复。

4.4 修复 protobuf 版本冲突 (如有需要)

如果上面验证时出现类似这样的错误:

```

ImportError: cannot import name 'builder' from
'google.protobuf.internal'

```

这是因为平台环境中安装的 `protobuf` 版本过旧 (通常低于 3.20), 而 `onnx==1.16.0` 需要 `protobuf>=3.20.1`。运行下方单元格升级即可修复, 不会影响其他功能。

```

In [ ]: print("升级 protobuf 到兼容版本...")

!pip install "protobuf>=3.20.1,<5" -q

print("\n再次验证: ")
!python -c "from chatterbox.tts import ChatterboxTTS; print('✅ 现在 chatterbox 可以

```

五、配置 TTS 服务器

Chatterbox TTS Server 通过 `config.yaml` 文件控制各种行为。我们需要修改其中的几个关键参数, 确保服务器在 AMD GPU 上正确运行。

需要修改的参数：

- `model_selector: chatterbox`：选择使用原版 Chatterbox 模型。另一个选项是 `chatterbox-turbo`，速度更快，但需要安装额外的依赖包，本教程暂不涉及。
- `device: cuda`：指定运行设备。虽然我们用的是 AMD GPU，但由于 ROCm 兼容了 CUDA API，这里填 `cuda` 即可，PyTorch 会自动识别并调用 AMD GPU。
- `bf16: false`：关闭 BF16 (bfloat16) 半精度模式。BF16 可以减少显存占用并加速推理，但不是所有 GPU 都完全支持，关闭它可以保证兼容性。如果后续发现速度较慢且显存充裕，可以将此改为 `true`。

In [15]:

```
import yaml, os

config_path = os.path.join(REPO_DIR, "config.yaml")

with open(config_path) as f:
    config = yaml.safe_load(f)

engine = config.setdefault("tts_engine", {})
engine["model_selector"] = "chatterbox"
engine["device"] = "cuda"
engine["bf16"] = True
config.setdefault("model", {})[ "repo_id" ] = "ResembleAI/chatterbox"
with open(config_path, "w") as f:
    yaml.dump(config, f, default_flow_style=False, allow_unicode=True)

print("✅ config.yaml 已更新：")
print(f"    model_selector : {engine['model_selector']}")
print(f"    device          : {engine['device']}")
print(f"    bf16            : {engine['bf16']}")
```

```
✅ config.yaml 已更新:
model_selector : chatterbox
device          : cuda
bf16            : True
```

六、启动 TTS 服务器

现在可以启动 TTS 服务器了。服务器以后台子进程的形式运行，不会阻塞 Notebook 的执行，日志输出重定向到 `/tmp/chatterbox_tts.log` 文件。

6.1 启动进程

注意环境变量的配置：

- `HF_ENDPOINT` 指向镜像站，这样服务器在下载模型时会使用镜像
- `PYTHONUNBUFFERED=1` 禁用 Python 输出缓冲，确保日志能实时写入文件
- `HSA_OVERRIDE_GFX_VERSION` 默认注释掉。如果遇到 `HIP error: invalid device function` 报错，说明 ROCm 没有正确识别 GPU 架构，取消注释并设置对应值（RX 7900 系列填 `11.0.0`）可以解决

```
In [16]: import subprocess, os, time

LOG_FILE = "/tmp/chatterbox_tts.log"

# 先停止已有进程, 避免端口冲突
!kill -f server.py 2>/dev/null; sleep 2

env = {
    **os.environ,
    "HF_ENDPOINT" : "https://hf-mirror.com",
    "PYTHONUNBUFFERED" : "1",
    # 遇到 HIP error: invalid device function 时取消注释:
    # "HSA_OVERRIDE_GFX_VERSION": "11.0.0",
}

proc = subprocess.Popen(
    ["python", "server.py"],
    cwd=REPO_DIR,
    env=env,
    stdout=open(LOG_FILE, "w"),
    stderr=subprocess.STDOUT,
)

print(f"✅ TTS 服务器进程已启动")
print(f"    PID      : {proc.pid}")
print(f"    日志文件  : {LOG_FILE}")
print(f"    服务地址  : http://localhost:8004")
```

```
✅ TTS 服务器进程已启动
PID      : 7133
日志文件 : /tmp/chatterbox_tts.log
服务地址 : http://localhost:8004
```

6.2 等待服务器就绪

服务器启动后, 会自动做以下几件事:

1. **下载模型** (仅首次): 从 hf-mirror.com 下载 Chatterbox 模型权重, 约 2-3 GB, 需要几分钟
2. **加载模型**: 将模型权重加载到 GPU 显存中
3. **启动 HTTP 服务**: 监听 8004 端口, 等待 API 请求

下方代码每隔 15 秒检查一次服务是否响应, 并打印最新的日志行, 方便你了解当前进度。日志中的百分比数字 (如 50%) 表示模型下载进度。

```
In [17]: import time, requests

MAX_WAIT = 900 # 最多等待 15 分钟
INTERVAL = 15 # 每 15 秒检查一次

def server_ready():
    try:
        return requests.get("http://localhost:8004/api/model-info", timeout=5).stat
```

```

except:
    return False

def last_log_line():
    try:
        with open(LOG_FILE) as f:
            lines = [l.strip() for l in f.readlines() if l.strip()]
            return lines[-1] if lines else " (日志为空) "
    except:
        return " (读取日志失败) "

print(f"等待服务器就绪 (最长等待 {MAX_WAIT // 60} 分钟) ...")
print("首次运行需要下载模型, 日志中出现百分比代表下载进度。\\n")

t0 = time.time()
while time.time() - t0 < MAX_WAIT:
    elapsed = int(time.time() - t0)
    if server_ready():
        print(f"\\n✅ 服务器就绪! (用时 {elapsed} 秒) ")
        break
    print(f"\\n[{elapsed:4d}s] {last_log_line()[115]}")
    time.sleep(INTERVAL)
else:
    print("\\n⚠️ 等待超时, 请查看完整日志: ")
    with open(LOG_FILE) as f:
        print(f.read()[-3000:])

```

等待服务器就绪 (最长等待 15 分钟) ...

首次运行需要下载模型, 日志中出现百分比代表下载进度。

[0s] 2026-06-12 08:32:31 [INFO] engine: Model type: original

✅ 服务器就绪! (用时 15 秒)

七、验证 GPU 加速

服务器就绪后, 查询服务器的模型信息, 确认模型正在 GPU 上运行, 而不是回退到 CPU 模式。

GPU 加速的判断依据是 PyTorch 的显存占用。加载 Chatterbox 模型后, GPU 显存通常会增加 2-5 GB。如果显存占用接近 0, 说明模型运行在 CPU 上, 需要检查配置。

```

In [18]: import requests

print("--- 服务器模型信息 ---")
try:
    info = requests.get("http://localhost:8004/api/model-info").json()
    for k, v in info.items():
        print(f" {k}: {v}")
except Exception as e:
    print(f"⚠️ 无法获取服务器信息: {e}")
    print(" 请先确认服务器已通过第六步启动。")

```

```
--- 服务器模型信息 ---
loaded: True
type: original
class_name: ChatterboxTTS
device: cuda
sample_rate: 24000
supports_paralinguistic_tags: False
available_paralinguistic_tags: []
turbo_available_in_package: False
multilingual_available_in_package: False
supports_multilingual: False
supported_languages: {'en': 'English'}
```

从启动日志中提取与 GPU 和模型相关的关键行，可以快速确认设备是否正确选择。

```
In [19]: print("--- 启动日志中的 GPU/模型信息 ---")
try:
    with open("/tmp/chatterbox_tts.log") as f:
        for line in f:
            line = line.strip()
            if any(k in line for k in ["device", "cuda", "ROCM", "GPU", "BF16", "lo
                print(f" {line}")
except:
    print(" (日志文件不存在) ")
```

```

--- 启动日志中的 GPU/模型信息 ---
2026-06-12 08:32:31 [INFO] __main__: Starting TTS Server directly on http://0.0.0.0:8004
2026-06-12 08:32:31 [INFO] __main__: API documentation will be available at http://0.0.0.0:8004/docs
2026-06-12 08:32:31 [INFO] __main__: Web UI will be available at http://0.0.0.0:8004/
INFO:      Started server process [7133]
INFO:      Waiting for application startup.
2026-06-12 08:32:31 [INFO] server: TTS Server: Initializing application...
2026-06-12 08:32:31 [INFO] server: Configuration loaded. Log file at: /workspace/repo/logs/tts_server.log
2026-06-12 08:32:31 [INFO] engine: CUDA requested and functional. Using CUDA.
2026-06-12 08:32:31 [INFO] engine: Final device selection: cuda
2026-06-12 08:32:31 [INFO] engine: BF16 optimization: disabled (TTS_BF16=off)
2026-06-12 08:32:31 [INFO] engine: Model selector from config: 'ResembleAI/chatterbox'
2026-06-12 08:32:31 [INFO] engine: Model selector 'ResembleAI/chatterbox' resolved to Original model (ChatterboxTTS)
2026-06-12 08:32:31 [INFO] engine: Initializing ChatterboxTTS on device 'cuda'...
2026-06-12 08:32:31 [INFO] engine: Model type: original
2026-06-12 08:32:43 [INFO] root: input frame rate=25
loaded PerthNet (Implicit) at step 250,000
2026-06-12 08:32:45 [INFO] engine: BF16 optimization disabled (TTS_BF16=off or hardware unsupported).
2026-06-12 08:32:45 [INFO] engine: Successfully loaded ChatterboxTTS on cuda
2026-06-12 08:32:45 [INFO] engine: Model sample rate: 24000 Hz
2026-06-12 08:32:45 [INFO] engine: TTS Model loaded successfully on cuda. Engine sample rate: 24000 Hz.
2026-06-12 08:32:45 [INFO] server: TTS Model loaded successfully via engine.
2026-06-12 08:32:45 [INFO] server: Application startup sequence complete.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8004 (Press CTRL+C to quit)
2026-06-12 08:32:46 [INFO] server: Attempting to open web browser to: http://localhost:8004/

```

八、生成第一段语音

服务器提供 OpenAI 兼容的 `/v1/audio/speech` 接口，请求格式与 OpenAI 的 TTS API 完全一致，因此任何支持 OpenAI TTS 的客户端都可以直接接入。

API 的核心参数：

- `model`：填 `tts-1`（兼容写法，服务器内部会忽略此字段，使用 `config.yaml` 中配置的模型）
- `input`：需要合成的文本
- `voice`：声音文件名，必须是服务器中已存在的预置声音文件

首先获取所有可用声音的列表：

```

In [20]: import requests

resp = requests.get("http://localhost:8004/v1/audio/voices").json()

```

```

voices = resp.get("voices", [])
print(f"可用声音 (共 {len(voices)} 个): ")
print("-" * 44)
for i, v in enumerate(voices):
    end = "\n" if (i + 1) % 4 == 0 else " "
    print(f"{i+1:2d}. {v[:<18]}", end=end)
print()

```

可用声音 (共 28 个):

```

-----
1. Abigail.wav      2. Adrian.wav      3. Alexander.wav   4. Alice.wa
v
5. Austin.wav       6. Axel.wav        7. Connor.wav      8. Cora.wav
9. Elena.wav        10. Eli.wav        11. Emily.wav       12. Everett.
wav
13. Gabriel.wav     14. Gianna.wav     15. Henry.wav       16. Ian.wav
17. Jade.wav        18. Jeremiah.wav   19. Jordan.wav      20. Julian.w
av
21. Layla.wav       22. Leonardo.wav   23. Michael.wav     24. Miles.wa
v
25. Olivia.wav      26. Ryan.wav       27. Taylor.wav      28. Thomas.w
av

```

从上方列表中选一个声音，输入想要合成的文本，然后运行下方代码。生成完成后会自动在 Notebook 中显示音频播放器。首次时间较长，ROCm 需要编译 GPU 内核 (kernel compilation)，生成对应显卡的优化代码。

```

In [21]: import requests, IPython.display as ipd, time

TEXT = "Hello! This is Chatterbox TTS running on AMD GPU with ROCm acceleration."
VOICE = "Alice.wav" # 修改为上方列表中的任意声音

print(f"合成中... 声音: {VOICE}")
print(f"文本: {TEXT}")

t0 = time.time()
r = requests.post(
    "http://localhost:8004/v1/audio/speech",
    json={"model": "tts-1", "input": TEXT, "voice": VOICE},
    timeout=120
)

print(f"\n耗时: {time.time()-t0:.1f} 秒 | 状态码: {r.status_code} | 大小: {len(r.

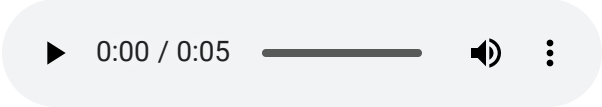
if r.status_code == 200:
    print("\n✅ 合成成功! 点击下方播放器收听: ")
    ipd.display(ipd.Audio(r.content, autoplay=True))
else:
    print("\n❌ 合成失败: ", r.text)

```


合成中... 声音: Alice.wav
文本: Hello! This is Chatterbox TTS running on AMD GPU with ROCm acceleration.

耗时: 18.0 秒 | 状态码: 200 | 大小: 284,204 字节

✅ 合成成功! 点击下方播放器收听:



九、交互式语音合成控制面板

每次修改代码、重新运行单元格的方式不够直观。下面使用 `ipywidgets` 构建一个可视化的控制面板, 让你可以随时调整参数、即时试听效果, 而不需要碰代码。

`ipywidgets` 是 Jupyter 官方的交互控件库, 支持按钮、滑块、下拉菜单等常见 UI 组件, 非常适合在 Notebook 里构建简单的演示界面。

面板参数说明:

参数	含义	推荐范围
随机性 (Temperature)	控制语音的随机变化程度。值越高, 每次生成的语音变化越大; 值越低, 每次结果越接近。	0.5 ~ 1.0
夸张度 (Exaggeration)	控制情感表达的强度。值越高, 语气越夸张、富有感情; 值越低, 语气越平淡、中性。	0.3 ~ 0.7

运行下方单元格, 控制面板会在输出区域出现:

```
In [22]: import ipywidgets as widgets
import requests, IPython.display as ipd, os, time

try:
    voices = requests.get("http://localhost:8004/v1/audio/voices").json().get("voices")
    if not voices:
        raise ValueError("空列表")
except Exception as e:
    voices = ["Alice.wav", "Ryan.wav", "Emily.wav", "Jordan.wav"]

header = widgets.HTML(
    "<h3 style='margin:0 0 6px 0; color:#cc0000;'>🔊 Chatterbox TTS — AMD ROCm GPU"
)
text_input = widgets.Textarea(
    value="Welcome to Chatterbox TTS, powered by AMD ROCm GPU acceleration.",
    placeholder="在此输入要合成的文本...",
    layout=widgets.Layout(width="100%", height="75px")
)
voice_dd = widgets.Dropdown(
    options=voices, value=voices[0] if voices else None,
```

```

        description="声音: ", style={"description_width": "50px"},
        layout=widgets.Layout(width="280px")
    )
    temp_sl = widgets.FloatSlider(
        value=0.8, min=0.1, max=1.5, step=0.05, description="随机性: ",
        style={"description_width": "65px"}, readout_format=".2f",
        layout=widgets.Layout(width="400px")
    )
    exag_sl = widgets.FloatSlider(
        value=0.5, min=0.0, max=1.0, step=0.05, description="夸张度: ",
        style={"description_width": "65px"}, readout_format=".2f",
        layout=widgets.Layout(width="400px")
    )
    gen_btn = widgets.Button(description=" 生成语音", button_style="primary", icon="play")
    save_btn = widgets.Button(description=" 保存 WAV", button_style="success", icon="document")
    status = widgets.Label(value="就绪, 点击「生成语音」开始合成。")
    out = widgets.Output()
    _state = {"audio": None, "n": 0}

    def on_gen(b):
        gen_btn.disabled = True; save_btn.disabled = True
        status.value = "⌚ 正在合成..."
        with out:
            out.clear_output(wait=True)
            try:
                t0 = time.time()
                r = requests.post(
                    "http://localhost:8004/v1/audio/speech",
                    json={"model": "tts-1", "input": text_input.value, "voice": voice_dropdown.value},
                    timeout=120
                )
                elapsed = time.time() - t0
                if r.status_code == 200:
                    _state["audio"] = r.content; _state["n"] += 1; save_btn.disabled = False
                    status.value = f"✅ 完成 {len(r.content):,} 字节 · {elapsed:.1f}s"
                    ipd.display(ipd.Audio(r.content, autoplay=True))
                else:
                    status.value = f"❌ 失败 HTTP {r.status_code}"; print(r.text[:200])
            except requests.exceptions.Timeout:
                status.value = "❌ 超时 (文本过长或服务器繁忙)"
            except Exception as e:
                status.value = f"❌ 错误: {str(e)[:60]}"
        gen_btn.disabled = False

    def on_save(b):
        if _state["audio"]:
            path = f"/workspace/tts_{_state['n']:03d}.wav"
            open(path, "wb").write(_state["audio"])
            status.value = f"💾 已保存: {path}"

    gen_btn.on_click(on_gen); save_btn.on_click(on_save)

    ipd.display(widgets.VBox([
        header,
        widgets.HTML("<hr style='margin:6px 0'>"),
        text_input,

```

```

widgets.HBox([voice_dd], layout=widgets.Layout(margin="6px 0")),
temp_sl, exag_sl,
widgets.HBox([gen_btn, widgets.HTML("&nbsp;&nbsp;&nbsp;"), save_btn], layout=widgets.
status, out,
], layout=widgets.Layout(padding="14px", border="1px solid #ddd", border_radius="8p

```

VBox(children=(HTML(value="<h3 style='margin:0 0 6px 0; color:#cc0000;'> 🗣 Chatterbox TTS – AMD ROCm GPU</h3>"...

十、批量生成语音

在实际使用场景中，我们往往需要一次处理多段文本，例如将一本书的每个章节分别合成为独立的音频文件。

下方代码展示了批量合成的基本流程：循环遍历文本列表，逐一调用 API，将结果保存为编号的 WAV 文件，并统计总耗时。

你可以修改 `TEXTS` 列表和 `VOICE` 变量来自定义批量合成的内容：

```

In [23]: import requests, IPython.display as ipd, os, time

TEXTS = [
    "Welcome to the AMD ROCm developer platform.",
    "Chatterbox TTS delivers high quality, natural-sounding voice synthesis.",
    "This notebook runs entirely on AMD GPU hardware with ROCm acceleration.",
    "The Chatterbox model supports multiple voices and voice cloning features.",
]
VOICE      = "Ryan.wav"
OUTPUT_DIR = "/workspace/tts_batch"

os.makedirs(OUTPUT_DIR, exist_ok=True)
print(f"开始批量合成: {len(TEXTS)} 段文本, 声音: {VOICE}")
print(f"输出目录: {OUTPUT_DIR}\n")

t_total = time.time()
ok = 0

for i, text in enumerate(TEXTS):
    print(f"[{i+1}/{len(TEXTS)}] {text[:55]}...")
    t0 = time.time()
    r = requests.post(
        "http://localhost:8004/v1/audio/speech",
        json={"model": "tts-1", "input": text, "voice": VOICE},
        timeout=120
    )
    if r.status_code == 200:
        path = os.path.join(OUTPUT_DIR, f"clip_{i+1:02d}.wav")
        open(path, "wb").write(r.content)
        print(f"✅ {path} ({len(r.content):,} 字节, {time.time()-t0:.1f}s)")
        ipd.display(ipd.Audio(r.content))
        ok += 1
    else:
        print(f"❌ HTTP {r.status_code}: {r.text[:100]}")

```

```
print(f"\n完成: {ok}/{len(TEXTS)} 段成功 总耗时: {time.time()-t_total:.1f} 秒")
```

开始批量合成: 4 段文本, 声音: Ryan.wav

输出目录: /workspace/tts_batch

[1/4] Welcome to the AMD ROCm developer platform....

✓ /workspace/tts_batch/clip_01.wav (113,324 字节, 5.4s)

▶ 0:00 / 0:02 ————— 🔊 ⋮

[2/4] Chatterbox TTS delivers high quality, natural-sounding ...

✓ /workspace/tts_batch/clip_02.wav (180,524 字节, 7.5s)

▶ 0:00 / 0:03 ————— 🔊 ⋮

[3/4] This notebook runs entirely on AMD GPU hardware with RO...

✓ /workspace/tts_batch/clip_03.wav (217,004 字节, 8.8s)

▶ 0:00 / 0:04 ————— 🔊 ⋮

[4/4] The Chatterbox model supports multiple voices and voice...

✓ /workspace/tts_batch/clip_04.wav (197,804 字节, 8.2s)

▶ 0:00 / 0:04 ————— 🔊 ⋮

完成: 4/4 段成功 总耗时: 29.9 秒

十一、GPU 性能监控

了解模型在 GPU 上的资源占用情况, 有助于判断性能瓶颈和优化方向。在语音生成的同时打开 Terminal 输入 `watch -n 2 amd-smi` 可以监控到 GPU 内存变化。

`rocm-smi` 可以查看更底层的显存信息, 以及 GPU 的计算核心利用率 (类似 CPU 的使用率)。在进行语音合成请求时运行这个单元格, 可以看到 GPU 利用率的变化。(建议在终端运行 `rocm-smi`)

```
In [37]: print("=== rocm-smi 显存信息 ===")
!rocm-smi --showmeminfo vram

print("\n=== GPU 计算利用率 ===")
!rocm-smi --showuse
```

```
=== rocm-smi 显存信息 ===
```

```
===== ROCm System Management Interface =====
=====
===== Memory Usage (Bytes) =====
=====
GPU[0]          : VRAM Total Memory (B): 51522830336
GPU[0]          : VRAM Total Used Memory (B): 6407266304
=====
=====
===== End of ROCm SMI Log =====
=====
```

```
=== GPU 计算利用率 ===
```

```
===== ROCm System Management Interface =====
=====
===== % time GPU is busy =====
=====
GPU[0]          : GPU use (%): 0
=====
=====
===== End of ROCm SMI Log =====
=====
```

十二、停止服务器

当不再需要 TTS 服务时，运行下方代码停止服务器进程。这会释放 GPU 显存，避免占用其他任务的资源。

如果你关闭了 Jupyter Kernel，服务器进程会随之终止；但如果只是关闭浏览器页面，进程可能仍在后台运行，这时需要手动停止。

```
In [43]: !kill -f server.py && echo '✅ TTS 服务器已停止, GPU 显存已释放.' || echo '❌ 服务
```

知识小测验

完成本教程后，用三道小题检验一下你的掌握情况。运行每个单元格即可作答。

问题 1

在 AMD OneClick 平台上，我们安装 `chatterbox-v2` 时加了 `--no-deps` 标志。这样做的主要原因是什么？

```
In [24]: import ipywidgets as widgets, IPython.display as ipd
```

```

options_q1 = [
    ("A. 加快安装速度, 跳过不必要的下载",
    ("B. 防止 pip 将 ROCm 版 PyTorch 替换为 CPU 版",
    ("C. chatterbox-v2 本身没有任何依赖, 加不加没区别",
    ("D. 避免与 onnx 版本产生冲突",
]

radio_q1 = widgets.RadioButtons(
    options=[o[0] for o in options_q1],
    description="", layout=widgets.Layout(width="600px")
)

btn_q1 = widgets.Button(description="提交答案", button_style="primary")
out_q1 = widgets.Output()

def check_q1(b):
    with out_q1:
        out_q1.clear_output()
        selected = radio_q1.value
        correct = next(o[0] for o in options_q1 if o[1])
        if selected == correct:
            print("✅ 正确!")
            print(" chatterbox-v2 在 PyPI 上声明依赖的是 CPU 版 PyTorch。")
            print(" 不加 --no-deps, pip 会自动安装 CPU 版 torch 并覆盖已有的 ROCm 版")
            print(" 导致 GPU 加速完全失效, 模型只能在 CPU 上运行。")
        else:
            print(f"❌ 不对, 再想想。提示: 思考 chatterbox-v2 的 PyPI 元数据里声明了什

btn_q1.on_click(check_q1)
ipd.display(widgets.VBox([radio_q1, btn_q1, out_q1]))

```

VBox(children=(RadioButtons(layout=Layout(width='600px'), options=('A. 加快安装速度, 跳过不必要的下载', 'B. 防止 pip 将 ROCm 版 P...

问题 2

在本教程的 `config.yaml` 中, 我们将 `device` 设置为 `cuda`, 而不是 `amd` 或 `rocm`。
为什么 AMD GPU 要用 `cuda` 这个值?

```

In [25]: import ipywidgets as widgets, IPython.display as ipd

options_q2 = [
    ("A. config.yaml 是从 NVIDIA 版本复制来的, 忘记改了",
    ("B. AMD GPU 不被 PyTorch 支持, 只能用 CUDA 模式模拟",
    ("C. ROCm 版 PyTorch 实现了与 CUDA 兼容的 API, cuda.is_available() 对 AMD GPU 也
    ("D. cuda 是通用名称, 代表所有类型的 GPU",
]

radio_q2 = widgets.RadioButtons(
    options=[o[0] for o in options_q2],
    description="", layout=widgets.Layout(width="660px")
)

btn_q2 = widgets.Button(description="提交答案", button_style="primary")
out_q2 = widgets.Output()

def check_q2(b):

```

```

with out_q2:
    out_q2.clear_output()
    correct = next(o[0] for o in options_q2 if o[1])
    if radio_q2.value == correct:
        print("✅ 正确!")
        print("    AMD ROCm 在 PyTorch 层面实现了 CUDA 兼容 API (底层使用 HIP 运行)")
        print("    因此 torch.cuda.is_available()、torch.cuda.get_device_name() 等")
        print("    在 AMD GPU 上同样可用, 代码层面不需要区分 NVIDIA 和 AMD。")
    else:
        print("❌ 不对, 再想想。提示: 查看第七步验证 GPU 加速时的说明。")

btn_q2.on_click(check_q2)
ipd.display(widgets.VBox([radio_q2, btn_q2, out_q2]))

```

VBox(children=(RadioButtons(layout=Layout(width='660px'), options=('A. config.yaml
是从 NVIDIA 版本复制来的, 忘记改了', 'B....

问题 3

服务器第一次启动时速度很慢, 等了很长时间才就绪。下次重启服务器 (不删除缓存目录) 时, 速度会有明显变化吗? 为什么?

```

In [26]: import ipywidgets as widgets, IPython.display as ipd

options_q3 = [
    ("A. 不会, 每次启动都需要重新从 hf-mirror.com 下载模型", "False"),
    ("B. 会快很多, 模型权重已经下载到本地 HuggingFace 缓存目录, 直接读取即可", "True"),
    ("C. 会快一些, 但仍需验证远程服务器上的文件是否有更新", "False"),
    ("D. 取决于网速, 和第一次差不多", "False"),
]

radio_q3 = widgets.RadioButtons(
    options=[o[0] for o in options_q3],
    description="", layout=widgets.Layout(width="680px")
)
btn_q3 = widgets.Button(description="提交答案", button_style="primary")
out_q3 = widgets.Output()

def check_q3(b):
    with out_q3:
        out_q3.clear_output()
        correct = next(o[0] for o in options_q3 if o[1])
        if radio_q3.value == correct:
            print("✅ 正确!")
            print("    HuggingFace 库会将下载好的模型权重缓存到本地目录 (通常是 ~/.cache)")
            print("    再次启动时直接从磁盘加载, 跳过下载步骤, 等待时间从几分钟缩短到几十秒")
            print("    这也是为什么建议不要随意清空缓存目录。")
        else:
            print("❌ 不对, 再想想。提示: 思考 HuggingFace 的模型缓存机制。")

btn_q3.on_click(check_q3)
ipd.display(widgets.VBox([radio_q3, btn_q3, out_q3]))

```

VBox(children=(RadioButtons(layout=Layout(width='680px'), options=('A. 不会, 每次启动
都需要重新从 hf-mirror.com 下载模型', 'B. ...

常见问题排查

GPU 相关

问题	可能原因	解决方法
ROCM available: False	GPU 未被识别或驱动问题	检查 <code>rocminfo</code> 是否有输出；确认平台已分配 GPU
HIP error: invalid device function	GPU 架构与 ROCm 版本不匹配	在第六步的 <code>env</code> 中取消 <code>HSA_OVERRIDE_GFX_VERSION=11.0.0</code> 的注释
合成速度慢，显存占用为 0	模型运行在 CPU	检查 <code>config.yaml</code> 的 <code>device</code> 是否为 <code>cuda</code> ，重启服务器

安装相关

问题	可能原因	解决方法
triton wheel is not a supported wheel	Python 版本不符（轮子是 cp310，镜像是 py3.12）	跳过步骤 1（平台已预装 PyTorch），直接从步骤 2 开始
cannot import name 'builder' from protobuf	protobuf 版本过旧	运行第 4.4 节的修复单元格
chatterbox-v2 安装失败，找不到 ZIP	无 GitHub 访问且 ZIP 文件缺失	从本地下载 ZIP 上传到 <code>/workspace/</code> ，见第 4.3 节说明

服务器相关

问题	可能原因	解决方法
服务器长时间不就绪	首次运行需要下载 2-3 GB 模型	查看日志中的百分比，耐心等待，下载完成后会很快就绪
下载模型时连接超时	<code>HF_ENDPOINT</code> 未设置	确认第二步已运行，或手动 <code>export HF_ENDPOINT=https://hf-mirror.com</code>
声音文件 404 错误	<code>voice</code> 参数的文件名错误	先运行第八步的声音列表查询，使用正确的文件名
合成请求超时	文本过长或服务器 GPU 繁忙	将长文本分段（每段建议不超过 200 字）；减少并行请求

参考资源

- [Chatterbox TTS Server GitHub](#)
- [AMD ROCm AI 开发者中心](#)
- [AMD AI Playbooks \(官方实战教程\)](#)
- [ROCm 官方文档](#)
- [Hello ROCm 中文入门教程](#)
- [HuggingFace 镜像站 hf-mirror.com](#)
- [AMD OneClick Cloud 平台](#)